

In the script, you don't need to declare the type of a variable; it is inferred from the context. To make our life easier, helper functions like *pid*, *execname*, *log*, etc, are pre-defined. Look at the language reference guide for more information. If you have installed the package, you can find it at */usr/share/doc/systemtap-<version>/langref.pdf*.

How to run stap

The *stap* program can be invoked with multiple syntaxes:

```
stap -e 'script' [-c <target program>]
stap script.stp [-c <target program>]
stap -l 'event'>
```

Tapset libraries

In the example shown earlier, after probing on the read system call, we printed the name of the system call, and the arguments passed via name and argstr. This was possible because in one of the tapset libraries, */usr/share/systemtap/tapset/syscalls2.stp*, the following is defined:

```
probe syscall.read = kernel.function("Sys_read").call {,
    kernel.function("sys_read").call
{
    name = "read"
    fd = $fd
    buf_uaddr = $buf
    count = $count
    argstr = sprintf("%d, %d, %d", $fd, $buf, $count)
}
```

Tapsets provide abstraction to common probe points, and define functions that you can use in your script. They (probe aliases, not probes) are not runnable themselves.

Examples

```
$ cat syscount.stp
global syscalls
probe syscall.* { syscalls[name] += 1 }
probe timer.s(10) {
    foreach(n in syscalls- limit 5)
        printf("%s = %d\n", n, syscalls[n])
    delete syscalls
}
```

Here we have taken an associative array, *syscalls*. An associative array is a collection of unique keys – each key in the array has a value associated with it. Here, the name of each system call would be a unique key into the array. Whenever a system call is made, we increment the value of the element in the array that corresponds to the system call name. After 10 seconds, we print the top 5 system calls that were made.

```
$ stap syscount.stp
```

```
read = 116
poll = 55
ppoll = 49
setitimer = 24
writev = 22
```

Let's look at another script from which we want to get the process name and *pid* of the process that calls the maximum system calls. We also want to exclude the SystemTap process that launches the script (*stapio*) from consideration.

```
$ cat syscount_per_process.stp
global syscalls
probe syscall.* {
    if (execname() == "stapio")
        next
    syscalls[execname(), pid()] += 1
}
probe timer.s(10) {
    foreach(procname, id) in syscalls- limit 5
        printf("%s[%d] = %d\n", procname, id, syscalls[procname, id])
    delete syscalls
}
```

(To immediately return from a probe handler, we use the next statement.)

Running the script yields the following code:

```
$ stap syscount_per_process.stp
hald-addon-stor[1074] = 30
sendmail[1157] = 14
rtkit-daemon[1307] = 8
gdm-simple-gre[1374] = 8
gnome-power-man[1378] = 7
```

We can do other interesting stuff like aggregation, getting a call graph, and even modifying a kernel variable in the running kernel. We will cover this in next month's issue. 

References

<http://sourceware.org/systemtap/>
<http://sourceware.org/systemtap/wiki/LW2008SystemTapTutorial>
<http://sourceware.org/systemtap/wiki/HomePage?action=AttachFile&do=view&target=fosdem-stap.pdf>
<http://www.redbooks.ibm.com/redpapers/pdfs/redp4469.pdf>

By: Neependra Khare

The author is an open source enthusiast with a deep interest in Linux. He is currently working as a software engineer at KQ Infotech, and can be reached at neependra.khare@gmail.com. He also provides training on the Linux kernel and on debugging tools.